#TabularQ-Learning

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total	Q-table
LineWorld	1.00	5.0	0.0014 ms	10000	0.5 s	5
GridWorld	1.00	4.0	0.0018 ms	10000	0.8 s	23
TicTacToe	0.956	5.7	0.0028 ms	10000	0.6 s	1879
Quarto	0.166	20.5	0.0058 ms	10000	4.2 s	149422

Analyse

- Convergence immédiate sur petits environnements
- Très bon sur TicTacToe
- Mauvais sur Quarto

Comportement

- Adapté aux petits espaces d'états
- Explosion combinatoire

Points clés

- Ultra rapide
- Très efficace sur petits problèmes
- Non scalable

#DQN

> DQN apprend une fonction Q(s,a) qui estime la valeur de chaque action dans chaque état. À chaque step, l'agent choisit l'action avec la plus haute Q-valeur (exploitation) ou une action aléatoire avec une probabilité ε qui diminue au fil du temps (exploration). L'agent apprend à chaque step en minimisant l'erreur de Bellman : Q(s,a) doit tendre vers r + γ × max Q(s', a').

Sur LineWorld et GridWorld, l'algorithme apprend très vite. En seulement 1 000 épisodes, il atteint un score parfait (1.0). La loss devient rapidement proche de 0 et reste stable, ce qui montre que le modèle a bien appris.

Pour TicTacToe, l'apprentissage est rapide au début grâce à l'exploration (ε-greedy). Le score atteint 0.89 à 1 000 épisodes, puis 0.95 à 10 000. Ensuite, il baisse légèrement à 0.93 à 100

000 épisodes. Cette petite baisse vient du fait que DQN surestime parfois certaines actions, ce qui rend l'apprentissage un peu instable.

Quarto est beaucoup plus difficile. Le score monte à 0.17 à 10 000 épisodes puis redescend à 0.14 à 100 000. L'environnement est trop complexe (beaucoup d'actions, parties longues), donc le modèle a du mal à bien apprendre. Même si la loss diminue, les performances restent instables, ce qui montre que le modèle mémorise plus qu'il ne généralise.

Environnement	1k	10k	100k	Longueur	Temps/coup
LineWorld	1.00	1.00	1.00	5.00	0.03ms
GridWorld	1.00	1.00	1.00	4.00	0.03ms
TicTacToe	0.89	0.95	0.93	5.39	0.04ms
Quarto	0.10	0.17	0.14	22.52	0.04ms

#DoubleDQN

> Double DQN ajoute un réseau cible dont les poids sont figés et mis à jour périodiquement. La différence avec DQN est dans le calcul de la cible de Bellman :

```
• DQN        : target = r + γ × max Q_principal(s', a')
• Double DQN : target = r + γ × Q_cible(s', argmax Q_principal(s', a'))
Le réseau principal choisit l'action, le réseau cible l'évalue. Cela corrige
la surestimation des Q-valeurs observée dans DQN classique.
```

Sur LineWorld et GridWorld, rien de surprenant : DoubleDQN fait exactement pareil que DQN. Le score atteint 1.0 très rapidement, dès 1 000 épisodes. En fait, ces environnements sont tellement simples qu'on ne voit aucune différence entre les deux algorithmes.

Pour TicTacToe, c'est plus intéressant. DoubleDQN est clairement plus stable. Là où DQN commençait à régresser un peu avec le temps, DoubleDQN continue de s'améliorer. Le score passe de 0.93 à 0.96 entre 10 000 et 100 000 épisodes. C'est justement ce qu'on attend : le réseau cible aide à éviter les surestimations et rend l'apprentissage plus fiable sur le long terme.

Par contre, pour Quarto, ça ne marche pas très bien. Le score monte un peu au début (jusqu'à 0.23), puis il s'effondre à 0.08. On retrouve le même problème qu'avant : le jeu est trop complexe. Il y a trop d'actions, les parties sont longues, et sans outils supplémentaires, l'algorithme n'arrive pas vraiment à apprendre une bonne stratégie.

Algo	TicTacToe 1k	10k	100k	Quarto 1k	10k	100k
DQN	0.89	0.95	0.93	0.10	0.17	0.14
DoubleDQN	0.93	0.95	**0.96**	0.01	0.23	0.08

#DDQNER

> Avec **DDQN+ER**, on ajoute un *replay buffer*, c'est-à-dire une mémoire dans laquelle l'agent stocke ses expériences sous forme de transitions *(s, a, r, s', done)*.

- Plutôt que d'apprendre uniquement à partir de la dernière expérience vécue, l'agent pioche aléatoirement un ensemble de transitions dans cette mémoire pour s'entraîner. Cette approche permet de casser les corrélations temporelles entre les données et rend l'apprentissage beaucoup plus stable.
- Lors d'un premier test avec une capacité de buffer fixée à 500, un problème majeur est apparu sur GridWorld. Le score atteignait 1.0 après environ 1 000 épisodes, mais s'effondrait ensuite brutalement. En parallèle, la fonction de perte devenait totalement instable.
- Ce comportement s'explique par une capacité de buffer trop faible : l'agent recycle en boucle les mêmes transitions, ce qui entraîne une forme de sur-apprentissage. Résultat, les valeurs Q deviennent extrêmes et l'entraînement diverge.
- Après avoir augmenté la capacité du buffer à 5 000, le problème disparaît : l'apprentissage devient stable et GridWorld converge correctement.

Configuration	GridWorld 10k	Observation
buffer=500, batch=32	0.00	Instabilité — loss explose à 6×10^{22}
buffer=5000, batch=64	1.00	Convergence normale

Résultats (buffer=5 000)

Environnement	@1k	@10k	@100k	Longueur @100k	Temps/coup
LineWorld	1.00	1.00	1.00	5.00	0.03ms
GridWorld	1.00	1.00	1.00	4.00	0.03ms
TicTacToe	0.28	0.36	0.36	8.68	0.04ms
Quarto	-0.17	0.12	-0.04	23.02	0.04ms

LineWorld et GridWorld : convergence parfaite après correction du buffer.

TicTacToe : résultat décevant — le score plafonne à 0.36 bien en dessous de DQN (0.93) et DoubleDQN (0.96). La loss explose plusieurs fois jusqu'à 2.5×10^{11}. Le buffer de 5 000 est encore insuffisant pour TicTacToe qui nécessiterait un buffer plus grand et un lr plus petit.

Quarto : le score d'évaluation est négatif à 100k épisodes malgré un reward d'entraînement positif dans le graphique. Cette différence vient du fait que le graphique inclut l'exploration aléatoire (ε > 0) qui peut mener à des victoires par chance, tandis que l'évaluation (ε=0) montre la vraie policy apprise qui est mauvaise.

#DoubleDQNPrioritizedExperienceReplay (

> DDQN+PER améliore le replay buffer en donnant une priorité à chaque transition selon son erreur TD. Les transitions avec une grande erreur (l'agent est très surpris du résultat) sont tirées plus souvent pour l'apprentissage.

• DDQN+ER : toutes les transitions ont la même probabilité d'être tirées
• DDQN+PER : les transitions surprenantes sont tirées plus souvent → apprentissage plus efficace
La priorité d'une transition = |erreur TD| + epsilon pour éviter une priorité nulle.

Environnement	@1k	@10k	@100k	Longueur @100k	Temps/coup
LineWorld	1.00	1.00	1.00	5.00	0.03ms
GridWorld	—	—	—	—	—
TicTacToe	0.44	0.29	0.42	8.23	0.04ms
Quarto	-0.20	-0.20	-0.28	23.61	0.04ms

LineWorld : score parfait comme tous les autres algos.

GridWorld : non testé — DDQN+ER avait montré que ces algos avec buffer sont instables sur des épisodes courts. DQN et DoubleDQN donnent de meilleurs résultats sur GridWorld.

TicTacToe : le score oscille entre 0.29 et 0.44 sans converger clairement. DDQN+PER est légèrement meilleur que DDQN+ER (0.42 vs 0.36 à 100k) mais reste loin de DQN et DoubleDQN.

Quarto : résultat le plus décevant de la famille — le score est négatif et empire avec le temps (-0.20 à 1k, -0.28 à 100k). Les priorités mal estimées en début d'entraînement orientent l'agent vers de mauvaises transitions, ce qui nuit à l'apprentissage.

Résultats à 100 000 épisodes d'entraînement, évaluation sur 500 parties (ε=0). pour la famille DQN

TicTacToe

Algo	@1k	@10k	@100k	Verdict
DQN	0.89	0.95	0.93	Bon mais surestimation
DoubleDQN	0.93	0.95	0.96	Meilleur — stable sur le long terme
DDQN+ER	0.28	0.36	0.36	Décevant — buffer insuffisant
DDQN+PER	0.44	0.29	0.42	Instable — oscille sans converger

Quarto

Algo	@1k	@10k	@100k	Verdict
DQN	0.10	0.17	0.14	Instable
DoubleDQN	0.01	0.23	0.08	Instable
DDQN+ER	-0.17	0.12	-0.04	Très instable
DDQN+PER	-0.20	-0.20	-0.28	Le pire — empire avec le temps

Conclusion

- Sur les environnements simples (LineWorld, GridWorld), tous les algos convergent parfaitement dès 1 000 épisodes. La différence entre les algos n'est visible que sur les environnements complexes.
- Sur TicTacToe, DoubleDQN est le meilleur de la famille avec 0.96 à 100k épisodes. DDQN+ER et DDQN+PER donnent des résultats décevants car le buffer de 5 000 est insuffisant — ces algos ont besoin de beaucoup plus de données pour montrer leur avantage théorique.
- Sur Quarto, aucun algo de la famille DQN ne converge de manière stable. DDQN+PER donne même les pires résultats avec un score négatif qui empire. La complexité du jeu (32 actions, parties longues, récompense rare) dépasse les capacités de ces algorithmes sans techniques avancées comme le self-play ou un reward shaping.
- les algorithmes théoriquement plus puissants (DDQN+ER, DDQN+PER) ne donnent pas toujours de meilleurs résultats en pratique avec des ressources limitées. DQN et DoubleDQN restent les plus efficaces sur nos environnements avec les paramètres testés.

#REINFORCE

On joue sur les trois paramètres suivants
• gamma ($\gamma = 0.99$) : à quel point l'agent se soucie des récompenses futures. Proche de 1 =

pense long terme.
• lr (learning rate) : la taille des mises à jour. Trop grand = instable. Trop petit = apprend lentement.
• hidden_size : la taille du réseau de neurones. Plus grand = peut apprendre des stratégies plus complexes.

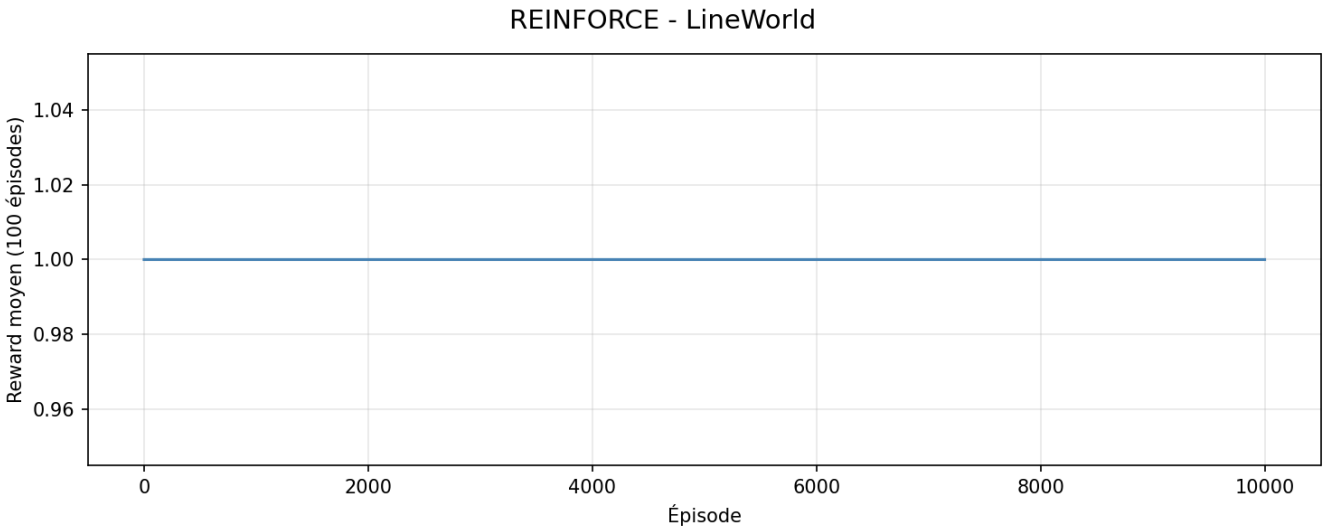
2. Résultats avec les paramètres de base

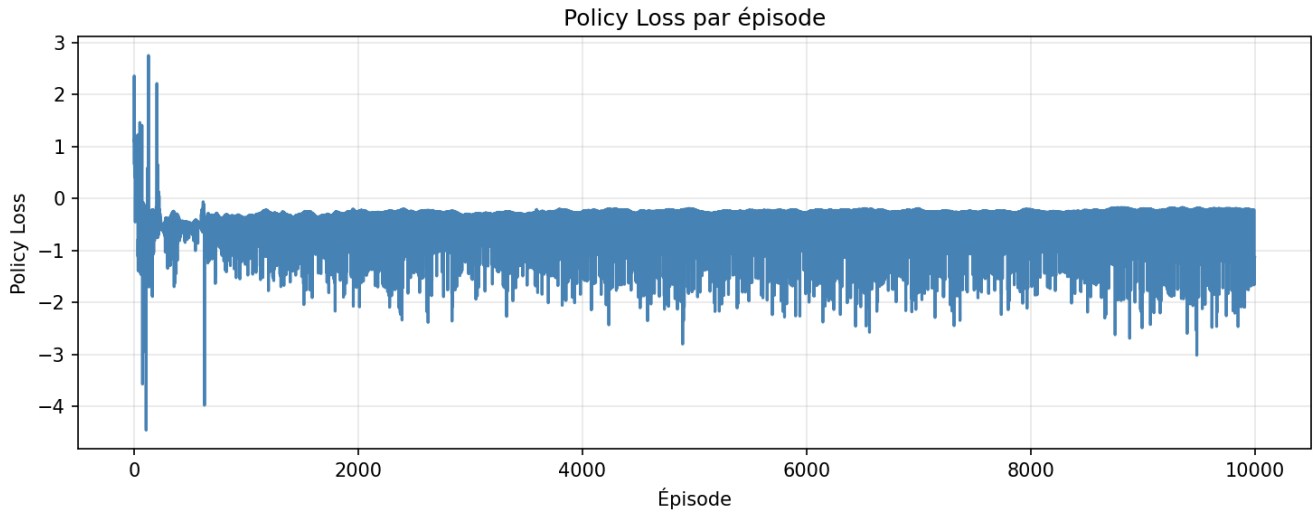
Configuration : gamma=0.99 · lr=0.001 · hidden_size=64 · 10 000 épisodes

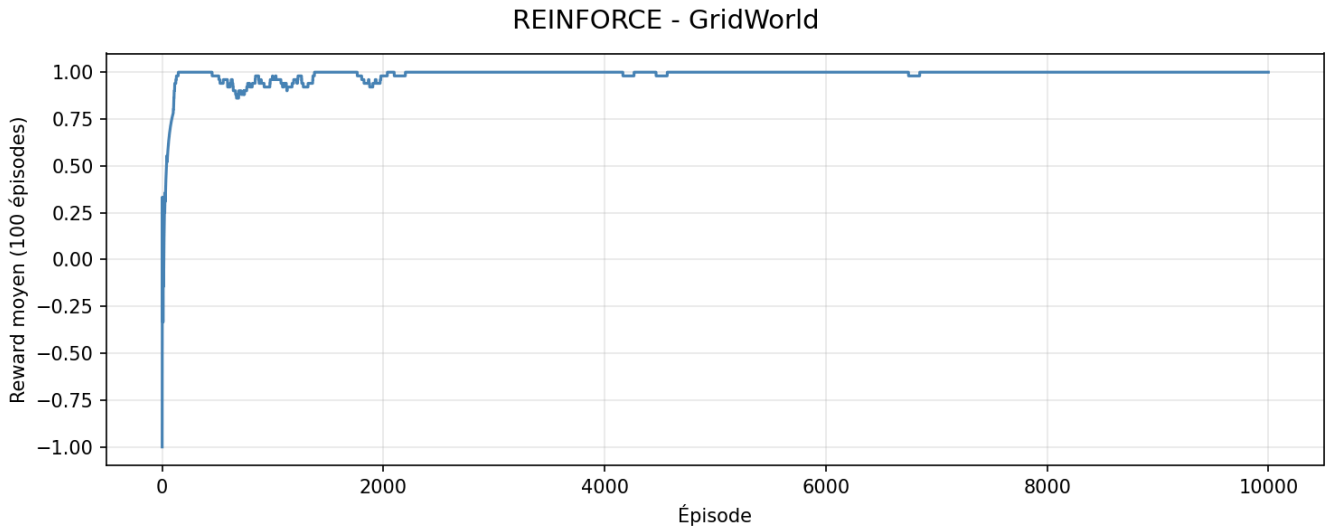
Environnement	Score 1 000 ep.	Score 10 000 ep.	Longueur moy.	Temps/coup	Conclusion
LineWorld	1.00	1.00	5 steps	0.04 ms	Parfait dès le départ
GridWorld	0.00	1.00	12 steps	0.04 ms	Converge à 10 000
TicTacToe	0.62	0.71	6 steps	0.04 ms	Apprend bien, peut mieux faire
Quarto	0.29	-0.01	23 steps	0.04 ms	Instable — retombe

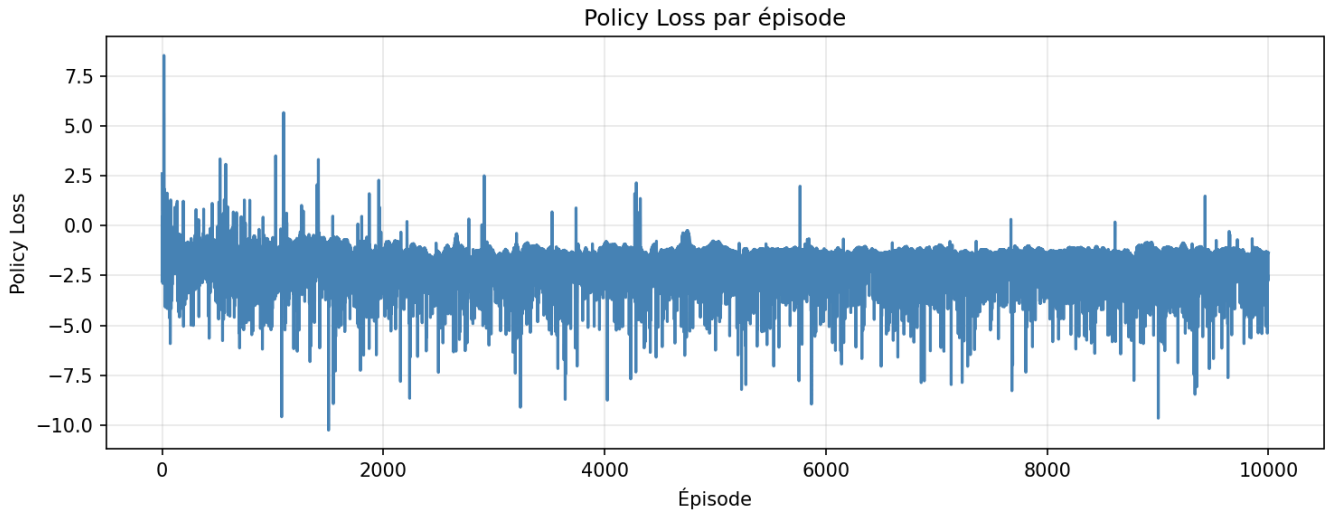
Observations :

• LineWorld et GridWorld : REINFORCE fonctionne parfaitement, les environnements sont simples.

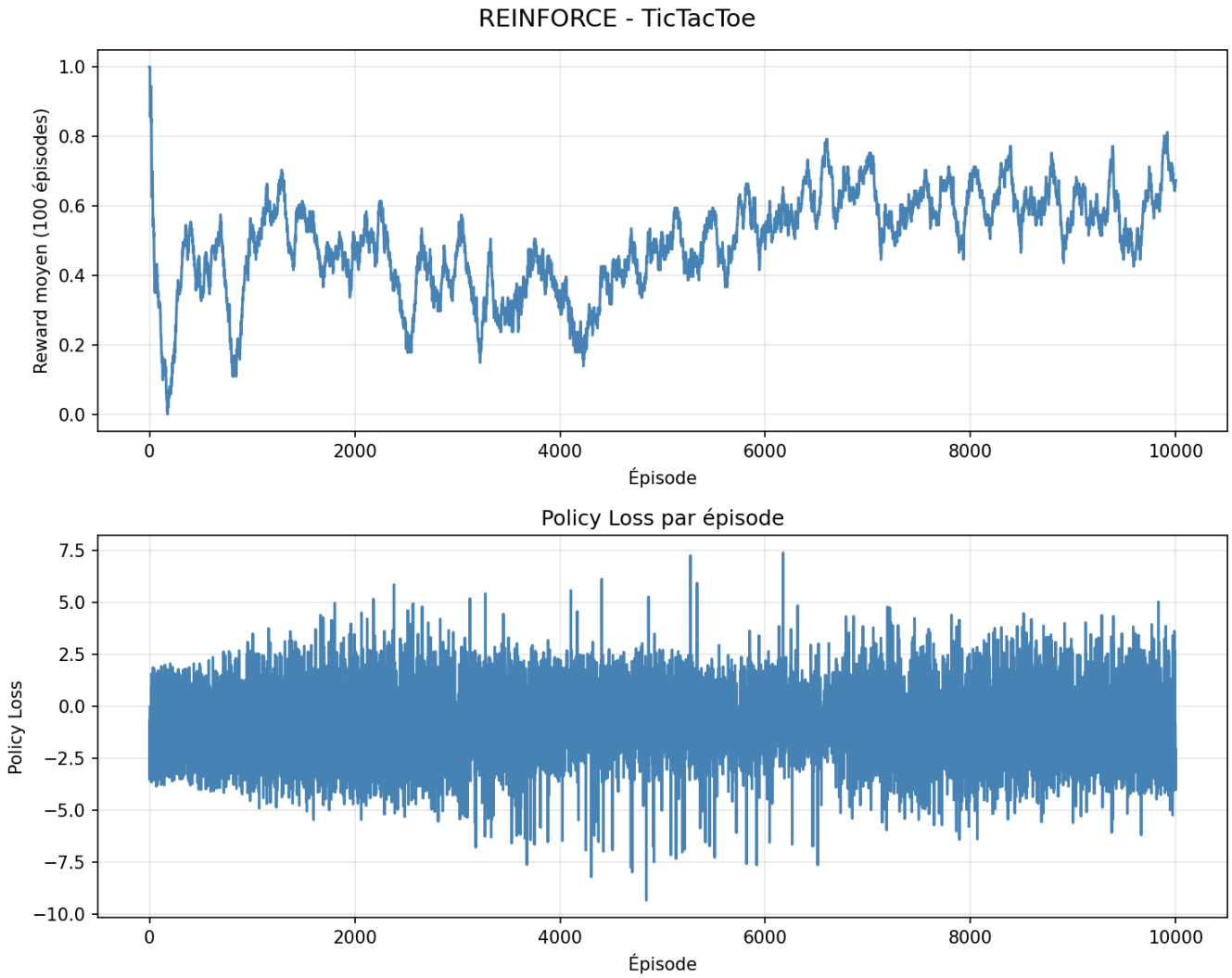

REINFORCE - LineWorld
Reward moyen (100 épisodes)
1.04
1.02
1.00
0.98
0.96
0
2000
4000
6000
8000
10000
Épisode


Policy Loss par épisode
Policy Loss
3
2
1
0
−1
−2
−3
−4
0
2000
4000
6000
8000
10000
Épisode



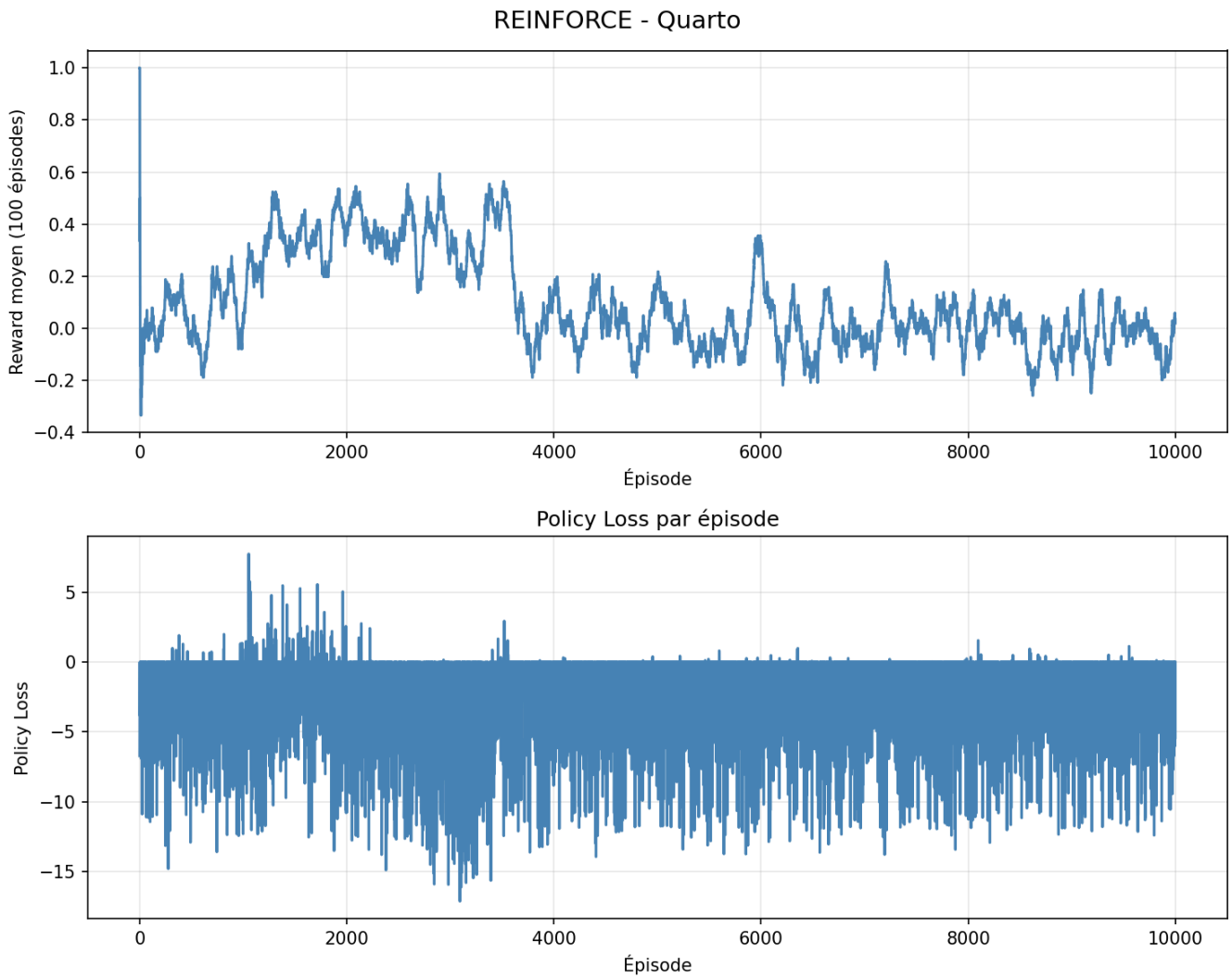


- TicTacToe : l'agent gagne 71% contre un joueur random, on peut faire mieux !!!



• Quarto : le score monte à 0.29 puis retombe à -0.01. C'est le signe que les mises à jour sont

trop grandes — l'agent désapprend ce qu'il a appris.



En regardant les graphiques, on identifie deux problèmes :

TicTacToe — le score progresse mais lentement et avec des oscillations. On a augmenté la taille du réseau de 64 à 128 pour avoir plus de capacité d'apprentissage et on a réduit le lr pour stabiliser.

Quarto — le score monte puis redescend (catastrophic forgetting). Avec 32 actions possibles et 105 dimensions d'état, les mises à jour à lr=0.001 sont trop brutales. On divise le lr par 10 et on double la taille du réseau.

Environnement	Paramètre	Avant	Après	Pourquoi
TicTacToe	lr	1e-3	5e-4	Réduire les oscillations
TicTacToe	hidden_size	64	128	Plus de capacité d'apprentissage
Quarto	lr	1e-3	1e-4	Éviter le catastrophic forgetting
Quarto	hidden_size	128	256	Mieux gérer 32 actions et 105 dims

On augmente également le nombre d'épisodes 100 000 épisodes pour observer la vraie progression.

Environnement	Config	Score 1k	Score 10k	Score 100k	Verdict
TicTacToe	lr=1e-3, h=64 (base)	0.62	0.71	—	Plafonné à 0.71
TicTacToe	lr=5e-4, h=128	0.70	0.78	~0.95	Excellent
Quarto	lr=1e-3, h=128 (base)	0.29	-0.01	—	Instable
Quarto	lr=1e-4, h=256	0.10	0.35	~0.10	Toujours instable

Pour Tictactoe, Le score monte de 0.25 jusqu'à presque 1.0 sur 100 000 épisodes de façon très régulière. C'est un résultat excellent, l'agent bat le joueur random dans presque toutes les parties.

Pour le quarto, le score monte jusqu'à 0.5 vers 15 000 épisodes puis redescend à nouveau. Même avec lr=1e-4 et hidden_size=256, REINFORCE n'est pas assez stable pour Quarto. La conclusion est claire .

Env	Config	1k	10k	100k	Longueur	Temps/coup	Bilan
LineWorld	base	1.00	1.00	1.00	5	0.04ms	Optimal
GridWorld	base	0.00	1.00	1.00	12	0.04ms	Convergé
TicTacToe	base (h=64)	0.62	0.71	—	6	0.04ms	Limité
TicTacToe	amélioré (h=128)	0.70	0.78	0.95	6	0.04ms	Excellent
Quarto	base (h=128)	0.29	-0.01	—	23	0.04ms	Instable
Quarto	amélioré (h=256)	0.10	0.35	0.10	23	0.04ms	Insuffisant

Conclusion

REINFORCE fonctionne très bien sur les environnements simples. Sur TicTacToe, avec les bons hyperparamètres et assez d'épisodes, il atteint un niveau quasi-parfait contre un joueur random.

En revanche, Quarto est trop complexe pour REINFORCE. Le problème principal est la variance élevée des gradients — sur 32 actions possibles, les mises à jour sont trop instables. La prochaine étape est de tester PPO A2C et REINFORCE with Critic qui apportent une baseline de valeur pour réduire cette variance.

#REINFORCEwithmeanbaseline

> Dans REINFORCE avec baseline, on enlève simplement la moyenne des récompenses obtenues, sans les normaliser complètement. Ça permet d'éviter des mises à jour trop grandes et rend l'apprentissage plus stable, surtout dans des environnements compliqués comme Quarto. En revanche, sur un jeu plus simple comme Tic-Tac-Toe, cette approche peut ralentir l'apprentissage. Dans ce cas, la version classique de REINFORCE, bien réglée, peut être plus efficace.

Algo	TicTacToe à 100k	Quarto à 100k
REINFORCE classique	**0.95**	0.10
REINFORCE Mean Baseline	0.73	**0.26**

#REINFORCEwithmeanCritic

Sur TicTacToe, REINFORCE with Critic est clairement le meilleur des trois — il apprend beaucoup plus vite (0.81 dès 1 000 épisodes) et atteint presque la perfection à 100 000 épisodes. Sur Quarto les trois algos se valent autour de 0.30 — aucun ne converge vraiment, ce qui confirme que Quarto nécessite peut être un algorithme plus avancé comme PPO.

Algo	TicTacToe 1k	10k	100k	Quarto 1k	10k	100k
REINFORCE	0.62	0.71	0.95	0.29	-0.01	0.10
Mean Baseline	0.25	0.71	0.73	0.12	0.30	0.26
Critic	**0.81**	**0.95**	**0.98**	0.14	0.34	**0.33**

#PPO

TicTacToe — excellent, 0.97 à 100k, légèrement en dessous de REINFORCE Critic (0.98) mais très proche. PPO apprend très vite dès le début , 0.79 à 1k contre 0.81 pour le Critic.

Quarto — comme tous les algos, ça plafonne autour de 0.20-0.33 avec des pics énormes dans la policy loss (jusqu'à 20 000 !), c'est le problème du ratio PPO qui peut exploser sur Quarto. Cela s'explique par la complexité du jeu et par le fait que le ratio utilisé par PPO peut parfois exploser, rendant l'apprentissage plus difficile malgré les mécanismes de stabilisation.

Algo	TicTacToe @100k	Quarto @100k
REINFORCE	0.95	0.10
Mean Baseline	0.73	0.26
Critic	**0.98**	0.33
PPO	0.97	0.21

#AlphaZero

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total
LineWorld	1.00	5.0	0.13 ms	5000	29 s
GridWorld	1.00	4.0	0.15 ms	5000	28 s
TicTacToe	0.95	5.88	0.20 ms	5000	56 s
Quarto	0.24	21.7	1.45 ms	5000	1284 s

Analyse

- Convergence immédiate sur LineWorld et GridWorld
- Performance optimale (score = 1.0)
- Apprentissage progressif sur TicTacToe
- Très bon winrate (~97%)
- Difficulté notable sur Quarto

Comportement

- Environnements simples → solution immédiate
- Environnement stratégique → apprentissage réel
- Environnement complexe (Quarto) → instabilité

Points clés

- Très rapide sur environnements simples
- Bonne généralisation
- MCTS coûteux pour peu de gain
- Temps d'entraînement élevé sur Quarto
- Difficulté avec grands espaces d’états

#ExpertApprentice

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total
LineWorld	1.00	5.0	0.15 ms	2000	310 s
GridWorld	1.00	4.0	0.48 ms	3000	459 s
TicTacToe	0.916	5.4	0.53 ms	5000	101 s
Quarto	0.21	21.4	0.21 ms	7000	540 s

Analyse

- Convergence immédiate sur tâches simples
- Bonne performance sur TicTacToe
- Difficulté sur Quarto

Comportement

- Très efficace sur tâches simples
- Apprentissage plus lent que AlphaZero
- Dépend fortement de MCTS

Points clés

- Temps d'entraînement élevé
- Moins robuste qu'AlphaZero
- Limité sur environnements complexes

#MCTS

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total
LineWorld	1.00	25.9	20.95 ms	1	517 s
GridWorld	1.00	5.47	71.57 ms	1	390 s
TicTacToe	0.982	5.65	10.65 ms	1	35 s
Quarto	0.842	18.24	33.92 ms	1	376 s

Analyse

- Excellentes performances sans entraînement
- Très efficace sur TicTacToe et Quarto

Comportement

- Basé sur simulation pure
- Recherche intensive à chaque coup
- Coût élevé

Points clés

- Aucun entraînement
- Très robuste
- Très lent
- Non scalable

#MuZeroStochastic

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total
LineWorld	1.00	5.0	0.40 ms	800	217 s
GridWorld	1.00	4.0	0.46 ms	800	863 s
TicTacToe	0.44	6.92	0.62 ms	800	188 s
Quarto	0.50	22.3	0.41 ms	300	392 s

Analyse

- Bon sur environnements simples
- Mauvais sur TicTacToe
- Instable sur Quarto

Comportement

- Modèle interne + MCTS
- Gère l'incertitude
- Sensible aux données

Points clés

- Bon potentiel
- Besoin de beaucoup d'épisodes
- Modèle complexe

#MuZero

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total
GridWorld	1.00	4.0	0.44 ms	100	8.6 s

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total
TicTacToe	0.40	7.3	0.48 ms	100	2.7 s
Quarto	0.20	20.8	0.49 ms	100	10.2 s

Analyse

- Très bon sur GridWorld
- Sous-apprentissage global
- Mauvais sur environnements complexes

Comportement

- Modèle + planification
- Besoin de beaucoup de données

Points clés

- Bon potentiel
- Sensible aux épisodes
- Moins performant qu'AlphaZero à faible budget

#RandomRollout

Environnement	Score	Longueur	Temps/coup	Épisodes	Temps total
GridWorld	1.00	7.4	168.23 ms	1	21.1 s
TicTacToe	0.90	6.2	36.86 ms	1	2.6 s
Quarto	1.00	17.2	364.44 ms	1	53.8 s

Analyse

- Bons résultats sans apprentissage
- Forte dépendance au hasard

Comportement

- Simulations aléatoires
- Pas de modèle

Points clés

- Aucun entraînement

- Peut surprendre
- Très coûteux en calcul
- Non scalable